Tema 4: BUCLES

Sitio: CAMPUS
Curso: Python - 2º DAW
Libro: Tema 4: BUCLES

Imprimido por: MARÍA HAIZHEUSKAYA
Día: jueves, 29 de octubre de 2026, 21:44

Tabla de contenidos

1. BUCLES

1. BUCLES

La iteración es uno de los pilares de la programación. Nos permite ejecutar un bloque de código múltiples veces sin tener que escribirlo repetidamente, optimizando así nuestros scripts y haciéndolos dinámicos.

1. Diferencia conceptual: Condicional vs Iteración

- **Condicional (`if`):** Evalúa una expresión lógica. Si es verdadera, ejecuta el bloque de código **una sola vez** y luego continúa con el resto del programa.
- **Iteración (`while` / `for`):** Evalúa una expresión lógica (o recorre una colección). Mientras la condición siga siendo verdadera o queden elementos por recorrer, el bloque de código se ejecuta **repetidas veces** (en bucle).

2. Concepto de bucle: for vs while

En Python, al igual que en Java, disponemos de dos estructuras principales para iterar, pero su uso semántico está mucho más diferenciado en Python:

- `while` **(Mientras):** Se utiliza cuando **no sabemos cuántas veces** se va a repetir el bucle. La iteración depende de que una condición cambie su estado a `False` en algún momento (cuidado con los bucles infinitos).
- `for` **(Para cada):** En Python, el `for` no es el clásico contador `(int i=0; i<10; i++)` de Java. Actúa como un *foreach*. Se utiliza cuando **sabemos exactamente la cantidad de repeticiones** o cuando queremos recorrer los elementos de una colección (listas, cadenas, diccionarios).

```
# Bucle While: Depende de una condición
contador = 0
while contador < 3:
    print("While iteración:", contador)
    contador += 1  # Es vital modificar la variable de control

# Bucle For: Recorre una colección directamente
tecnologias = ["HTML", "CSS", "Python"]
for tech in tecnologias:
    print("For iteración:", tech)
```

3. Uso de la función range()

Como Python no tiene el bucle `for` tradicional basado en contadores, utiliza la función `range()` para generar secuencias numéricas iterables al vuelo. Acepta hasta tres parámetros: `range(inicio, fin, salto)`.

```
# Genera números del 0 al 4 (el límite superior es excluyente)
for i in range(5):
    print(i)

# Rango desde el 2 hasta el 10, saltando de 2 en 2 (números pares)
for par in range(2, 11, 2):
    print(par)
```

4. Control de bucles: break, continue y else

Podemos alterar el flujo normal de un bucle usando ciertas palabras reservadas. Python añade una peculiaridad muy interesante: la cláusula `else` en los bucles.

- `break`**:** Rompe y finaliza el bucle inmediatamente.
- `continue`**:** Salta el resto del código en la iteración actual y pasa directamente a la siguiente repetición.
- `else` **(Exclusivo en Python):** El bloque `else` asociado a un bucle se ejecuta **solo si el bucle terminó de forma natural** (es decir, no fue interrumpido por un `break`). Es muy útil para operaciones de búsqueda.

```
# Ejemplo de búsqueda con For-Else
alumnos_daw = ["Ana", "Luis", "Carlos"]
buscado = "Pedro"

for alumno in alumnos_daw:
    if alumno == buscado:
        print("¡Alumno encontrado!")
        break
else:
    # Esto solo se ejecuta si el break NUNCA fue llamado
    print("El alumno no está matriculado.")
```

5. Bucles anidados

Podemos colocar un bucle dentro de otro. Es habitual al trabajar con matrices (listas de listas) o coordenadas bidimensionales. Por cada iteración del bucle externo, el bucle interno se ejecuta por completo.

```
coordenadas = [[1, 2], [3, 4], [5, 6]]

for fila in coordenadas:
    for valor in fila:
        print(f"Procesando valor: {valor}")
```

6. Bonus DAW: Comprensión de Listas (List Comprehensions)

Para alumnos avanzados de 2º DAW, es vital conocer que Python permite escribir un bucle `for` dentro de una lista para crear y filtrar colecciones en una sola línea de código, haciéndolo más eficiente y legible (estilo funcional).

```
numeros = [1, 2, 3, 4, 5, 6, 7, 8]

# Forma tradicional de sacar los pares
pares_tradicional = []
for n in numeros:
    if n % 2 == 0:
        pares_tradicional.append(n)

# Forma Pythonic (List Comprehension)
pares_python = [n for n in numeros if n % 2 == 0]

print(pares_python) # [2, 4, 6, 8]
```